ASSIGNMENT-19.4

2303A51600

BATCH-29

PROMPT:

"""Convert a JavaScript function that filters numbers greater than 50 and returns their average into Python using list

comprehension.

function getAverage(arr) {

let filtered = arr.filter(x => x > 50);

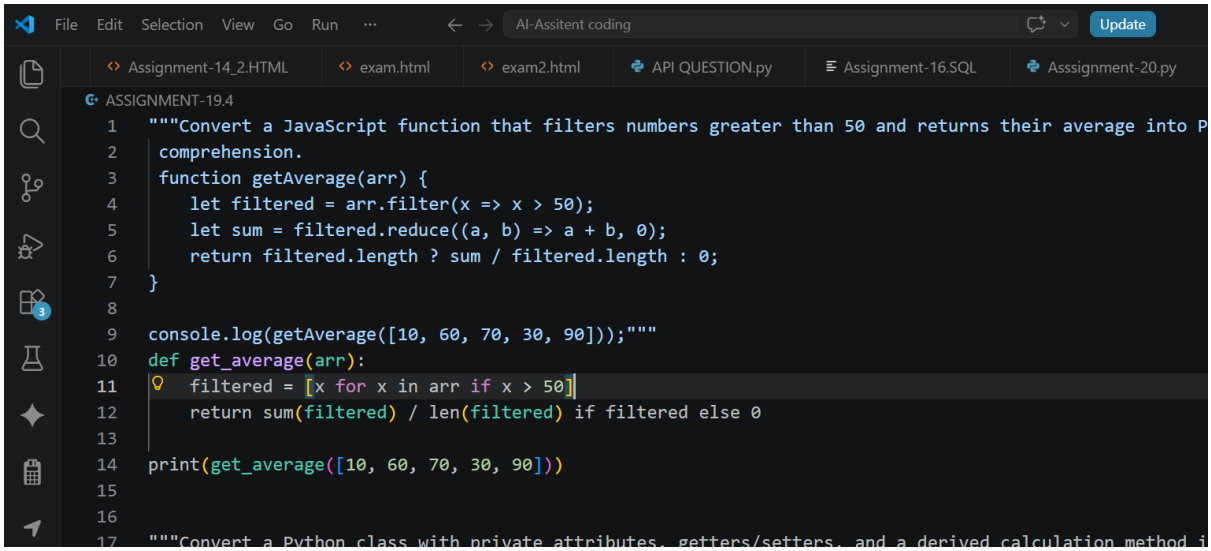
let sum = filtered.reduce((a, b) => a + b, 0);

return filtered.length ? sum / filtered.length : 0;

}

console.log(getAverage([10, 60, 70, 30, 90]));"""

CODE:



OBSERVATION:

Python replaces .filter() and .reduce() with **list comprehension and sum()**, making code shorter.

Both languages give the **same output**, but Python is more readable for this logic

PROMPT:

Convert a Python class with private attributes, getters/setters, and a derived calculation method into C++.

```
class Student:
  def __init__(self, marks):
    self.__marks = marks

  def get_marks(self):
    return self.__marks

  def set_marks(self, marks):
    self.__marks = marks

  def get_grade(self):
    return "Pass" if self.__marks >= 50 else "Fail"
```

CODE:

```
ASSIGNMENT-19.4
class Student:
    def get_grade(self):

#include <iostream>
using namespace std;

class Student {
private:
    int marks;

public:
    Student(int m) {
        marks = m;
    }

    int getMarks() {
        return marks;
    }

    void setMarks(int m) {
        marks = m;
    }

    string getGrade() {
        return marks >= 50 ? "Pass" : "Fail";
    }
};

int main() {
    Student s(60);
    cout << s.getGrade();
```

OBSERVATION:

Python uses __private (name mangling), while C++ uses **strict private access modifier**.

C++ requires **explicit data types and constructors**, unlike Python.

PROMPT:

"""Convert a Python GET API request using requests into JavaScript using Fetch API with async/await

import requests

try:

res = requests.get("https://jsonplaceholder.typicode.com/posts/1")

data = res.json()

print(data)

except Exception as e:

print(e)"""

CODE:

```
"""Convert a Python GET API request using requests into JavaScript using Fetch API with async/await
import requests

try:
    res = requests.get("https://jsonplaceholder.typicode.com/posts/1")
    data = res.json()
    print(data)
except Exception as e:
    print(e)"""
    async function fetchData() {
    try {
        let res = await fetch("https://jsonplaceholder.typicode.com/posts/1");
        let data = await res.json();
        console.log(data);
    } catch (err) {
        console.log(err);
    }
}

fetchData();
```

OBSERVATION:

Python uses requests, while JavaScript uses Fetch API with async/await.

Error handling differs: try-except vs try-catch.

PROMPT:

<!-- Convert an SQL query that groups by category, calculates average price, and filters results into MongoDB aggregation.

SELECT category, AVG(price) as avg_price

FROM products

GROUP BY category

HAVING AVG(price) > 100; -->

CODE:

```
<!-- Convert an SQL query that groups by category, calculates average price, and filters results into MongoDB aggregation

SELECT category, AVG(price) as avg_price
FROM products
GROUP BY category
HAVING AVG(price) > 100; -->

db.products.aggregate([
  {
    $group: {
      _id: "$category",
      avg_price: { $avg: "$price" }
    }
  },
  {
    $match: {
      avg_price: { $gt: 100 }
    }
  }
])
```

OBSERVATION:

SQL uses tables and GROUP BY, MongoDB uses JSON pipeline with $group.

MongoDB is more flexible but less structured than relational SQL.

PROMPT:

Implement binary search in Python, Java, and JavaScript ensuring consistent logic.

```
def binary_search(arr, target):
  l, r = 0, len(arr)-1
  while l <= r:
    m = (l+r)//2
    if arr[m] == target:
      return m
    elif arr[m] < target:
      l = m+1
    else:
      r = m-1
  return -1


print(binary_search([1,2,3,4,5], 4))
```

CODE:

Assignment-14_2.HTML | exam.html | exam2.html | API QUESTION.py | Assignment-1

ASSIGNMENT-19.4

```java
class Main {
    static int binarySearch(int arr[], int target) {
        int l = 0, r = arr.length - 1;
        while (l <= r) {
            int m = (l + r) / 2;
            if (arr[m] == target) return m;
            else if (arr[m] < target) l = m + 1;
            else r = m - 1;
        }
        return -1;
    }

    public static void main(String[] args) {
        int arr[] = {1,2,3,4,5};
        System.out.println(binarySearch(arr, 4));
    }
}

class Main {
    static int binarySearch(int arr[], int target) {
        int l = 0, r = arr.length - 1;
        while (l <= r) {
            int m = (l + r) / 2;
            if (arr[m] == target) return m;
            else if (arr[m] < target) l = m + 1;
            else r = m - 1;
        }
        return -1;
    }

    public static void main(String[] args) {
```

```java
class Main {
    static int binarySearch(int arr[], int target) {
        int l = 0, r = arr.length - 1;
        while (l <= r) {
            int m = (l + r) / 2;
            if (arr[m] == target) return m;
            else if (arr[m] < target) l = m + 1;
            else r = m - 1;
        }
        return -1;
    }

    public static void main(String[] args) {
        int arr[] = {1,2,3,4,5};
        System.out.println(binarySearch(arr, 4));
    }
}
```

```javascript
function binarySearch(arr, target) {
    let l = 0, r = arr.length - 1;
    while (l <= r) {
        let m = Math.floor((l + r) / 2);
        if (arr[m] === target) return m;
        else if (arr[m] < target) l = m + 1;
        else r = m - 1;
    }
    return -1;
}

console.log(binarySearch([1,2,3,4,5], 4));
```

OBSERVATION:

Core logic remains the same across all languages.

Differences are mainly in syntax and data type handling.